Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every buggy program triggers a fresh model call, so cost, latency, and external dependence scale with enrollment and recur every term, while the specification-level reasoning being purchased is identical across a class. Caching and cheaper models shrink only the constant, not the structure. We propose CoEP, which moves the LLM from the marginal-cost to the fixed-cost column: *spend the model once per assignment, not once per submission*. An LLM agent reads the assignment and a clustered sample of worked (buggy $\rightarrow$ fixed) pairs and authors the assignment-specific layer of a local repair tool: AST rewrite rules named after the misconceptions they fix, atop a shared symbolic substrate of generic edits and reference-guided minimal-hunk transplant from the class's own correct solutions. Every candidate patch is validated against the instructor's tests before it is kept; the tool can never self-report success, and submissions it abstains on escalate to a conventional per-submission call, so coverage is never sacrificed. On the standard benchmark of this literature, the synthesized tool repairs *more* submissions with *smaller* patches than the strongest hand-built symbolic tool, on that tool's own corpus and metric, with the entire model spend collapsed into a one-off synthesis plus a small escalation tail, so everything repaired locally costs no model call at all, offline and deterministically. A price-invariant break-even analysis shows when amortization wins and why cheaper models and caching do not change the answer, and the artifact yields what a per-query API cannot: reproducible minimal patches and a per-class misconception profile.

Introduction

Timely, individualized feedback is the scarcest resource in programming education: introductory courses enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions no instructor can examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy submission and the assignment's test suite, produce a small patch that makes it correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

Three generations of systems have pursued this goal. Early systems established the setting: test-driven repair whose patch must stay close to the student's program, because an unrelated rewrite passes the tests but teaches nothing (Yi et al. 2017). A second generation mined the class's own solution space, matching each buggy program against previously accepted solutions (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). This repairs cheaply, but it requires a large solution pool and fails whenever a submission matches no reference (Orvalho, Janota, and Manquinho 2025). LLM-based repair removed both constraints and now fixes most real student programs at modest per-bug prices (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024).

What the third generation fixed in coverage, it broke in cost structure. The deployment default is one model query per buggy submission, so spending grows with enrollment and restarts every term. The queries themselves are massively redundant. A single assignment is attempted by hundreds of students whose failures cluster into a few recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019), and on the corpus we study, 77% of a repair prompt is byte-identical across submissions to the same assignment. Paying a model to rediscover the same off-by-one for the three-hundredth time is not intelligence at work; it is a billing artifact. The standard remedies do not change this shape. Cheaper models and provider caching shrink the constant (Anthropic 2025; OpenAI 2024), and retrieval enriches each query (Dai et al. 2026), but every submission still triggers a model round trip with its marginal cost, latency, and data egress. The question is not how much a call costs, but where in the pipeline the LLM is invoked.

Our answer is to spend the LLM once per assignment, not once per submission. Agent research has shown that LLMs can author reusable code and amortize it across many uses (Cai et al. 2024; Wang et al. 2024), and classical synthesis has learned per-assignment fix rules from examples (Rolim et al. 2017). CoEP (Cost-Effective Program repair) instantiates this idea for assignment repair. An LLM agent reads the assignment once and authors its problem-specific layer: AST rewrite rules named after the misconceptions they fix.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

These rules run on a shared symbolic substrate, a generic edit library plus reference-guided transplant from the class's correct pool. Every candidate patch must pass the instructor's tests before it is kept, so a tool can never self-report success. The frozen tool then repairs submissions locally in about a second at zero marginal LLM cost, and the few it abstains on escalate to a conventional per-submission call.

The resulting artifact beats the hand-built state of the art on its own benchmark and metric: 97.8% repair rate at 0.326 mean relative patch size, against Refactory's 90.8% at 0.40, over 1,686 real student programs. We are explicit about attribution. The rule layer produces about 13% of local fixes, at near-minimal patch size and with a misconception label each; the transplant engine produces the rest, and an ablation shows the symbolic substrate alone already carries the coverage. The claim is therefore not that an LLM wrote a state-of-the-art repairer. The claim is the invocation structure: one cheap build plus a small fallback tail replaces an inference per submission, and the artifact it feeds exceeds the hand-built tool.

The stakes scale with deployment. In one course the per-submission bill is dollars; on an online judge it is not. The public Codeforces dump our benchmark draws from aggregates 12.6M submissions to over 10,000 problems (Penedo et al. 2025), and at a representative frontier tier, per-submission repair costs on the order of $8 per thousand submissions, re-paid as every new submission arrives. The comparison is also robust to pricing. If a build costs C_{build} and locally resolves a fraction $(1-f)$ of submissions that would each have cost Δ, amortization wins past $N^* = C_{\text{build}}/(\Delta(1-f))$, and N^* is invariant to token price when build and query spending follow the same schedule. The artifact also offers what per-submission inference cannot at any price. It assembles patches on the AST, so it cannot emit invalid code and never rewrites whole files. It is deterministic and runs offline, so student code never leaves the institution. Its rule-fired repairs aggregate into a per-class misconception profile that no API call returns.

This paper makes the following contributions:

- **A cost-structure reformulation of assignment repair.** We move the LLM from per-submission marginal cost to per-assignment fixed cost, with a price-invariant break-even analysis $N^* = C_{\text{build}}/(\Delta(1-f))$: amortization is a structural advantage, not a constant-factor one that cheaper models erase.
- **The CoEP system.** A builder agent synthesizes misconception-named AST rewrite rules through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023), atop a shared substrate of generic edits and reference-guided minimal-hunk transplant. Every patch is independently test-validated, and abstentions escalate to a strategy-matched LLM fallback.
- **An empirical result on the baseline's own corpus and metric.** CoEP's local repair exceeds Refactory on both coverage and patch size at negligible one-off build cost, with an honest mechanism attribution, a framework-only ablation isolating the LLM layer's contribution, and per-builder analyses across two commodity models.

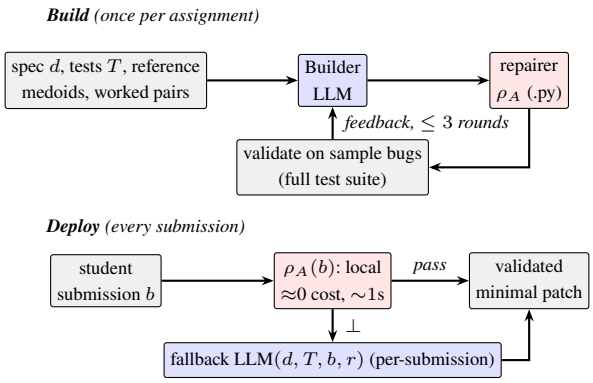


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, the large majority of submissions never touch a model.

- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

The Cost Anatomy of Per-Submission Repair

Decomposing the per-submission prompt of our PyDex reproduction shows where the money goes (Zhang et al. 2024; Xia and Zhang 2024). Averaged over the Refactory corpus, roughly three quarters of a query's input tokens are byte-identical across every submission to the same assignment: the description, the visible tests, and the fixed instructions. The buggy program is the only part that varies. Provider caching cannot even reach that redundancy. It discounts only a leading shared prefix, and PyDex's own prompt layout places the buggy program first, so no shared prefix precedes the variable content. Even reordered, these prompts fall below the providers' cacheable minimums of 1,024 to 4,096 tokens by model.

Two consequences follow. First, caching is a discount, not a fix, and here not even a discount: the redundancy is real, but provider caching serves it at the wrong granularity. Second, the redundancy is exactly the amortizable part. What is constant per assignment, namely understanding the specification, the tests, and the space of likely mistakes, can be reified once into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a test suite $T = \{(x_i, y_i)\}$ (function-entry or judge-style stdin/stdout), and the pool R of correct student solutions, which the setting assumes is available at feedback time, as every per-assignment system from CLARA to Refactory does (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). A *repair* of a buggy submission b is a program p that passes all of T. Patch quality is measured, never

[1] Anonymized repository submitted as supplementary material.

gated: RPS, the tree-edit distance between the ASTs of b and p normalized by the size of b (computed with Refactory's reference implementation (Hu et al. 2019)), is reported for every repair, since a small patch is feedback on the *student's* program while a rewrite passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$, an ordinary module with no model access, and deploys the hybrid: $\mathrm{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \perp$ (local, $\approx$0 marginal cost), else $\mathrm{LLM}(d, T, b, r)$: the conventional call, strengthened with one reference r chosen to match the student's strategy by clustering R on program structure and drawing r from the cluster nearest b.

Repairer Synthesis (once per assignment)

Build-set selection (local, no LLM cost). The builder does not see the whole pool; it sees a *chosen* sample. We cluster the buggy pool on a structural view (TF-IDF over normalized-AST paths, identifiers folded) and, walking clusters largest-first, take from each the member with the smallest tree-edit distance to its nearest correct solution, a clean single-misconception exemplar rather than the medoid's typical but tangled program, paired with that nearest correct program. The result is a set of worked (buggy $\rightarrow$ fixed) pairs: frequent misconceptions first, each shown *with* its minimal fix, so a weak builder needs only to generalize a transformation it is shown, not infer it.

Rule authoring. The builder LLM receives d, T, reference solutions (structural-cluster medoids of R), and the worked pairs, and emits the assignment-specific rule module: AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. The contract is strict. A rule locates its edit on the AST and splices it as a source-span replacement, never re-rendering the module, which would reformat untouched lines. It emits candidates rather than claimed fixes, and it never rewrites a whole function: a rule that cannot express a targeted edit must abstain so the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop: it loads the module, runs it on the sample bugs, *independently* validates every candidate patch by executing the full suite (the repairer's own claims are never trusted), and, while any sample remains unrepaired, returns the module with per-bug outcomes and the still-failing sources for revision, up to $R_{\max}$=3 builder calls. The best-validated round is kept, and build cost is exactly the token count of these exchanges, measured from the raw strings.

Local Repair (every submission)

The frozen tool repairs in three stages, cheapest and smallest-patch first, validating every candidate against the full suite before it is kept:

(1) Named rules + generic search. Rules are *candidate generators*, composed by a symptom-ordered combination search: the submission's failing-test signature ranks rules whose declared symptoms match, then single rules, then pairs, under an explicit candidate budget. A shared generic library of comparison flips, off-by-one range and index shifts, and Boolean-operator flips is always on, so common slips are covered regardless of what the builder wrote; it is the inversion of the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019). Every patch that survives carries the ordered list of rule names that produced it, provenance that aggregates into a per-class *misconception profile*.

(2) Reference-guided minimal-hunk transplant. Most student bugs need a structural edit no token-level rule expresses; the correct pool supplies it. We diff b against its structurally nearest correct solutions, rename the reference's variables to the student's (parameters by position, locals by first-appearance order), and adopt the *smallest* set of differing line-hunks that passes the tests, candidates ordered smallest-first and batch-validated, returning the minimum-RPS passer. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019), at hunk granularity rather than block granularity, which is what keeps patches smaller than the hand-built tool's.

(3) Escalation and self-learning. A submission both stages abstain on takes the fallback $\mathrm{LLM}(d, T, b, r)$, its cost counted against CoEP in every comparison. Each successful fallback fix can be generalized by the LLM into a new named rule, appended only if the augmented tool then repairs that miss on its own and rolled back otherwise, so the tool accumulates coverage across a term. Self-learning spends model calls only on submissions that already escalated, so the once-per-assignment build claim is untouched. To keep the accounting unambiguous, the frozen tool and the self-learning variant are evaluated and priced as separate configurations (Section).

The local path runs on a CPU in $\sim$1–2 s per submission, deterministically and offline; a defective tool degrades to $\perp$ (escalation), never to a wrong "repair", because the grading harness re-validates every patch before a student sees it.

Experiments

Setup

We compare six approaches: Refactory, PyDex, PyDex$_{\mathrm{PC}}$, and three CoEP configurations, all defined below. We ask three research questions. **RQ1 (Repair effectiveness):** how do the six approaches compare on per-problem and overall RR and RPS, and what do fallback and self-learning add on top of the local tool? **RQ2 (Cost):** what does each approach cost in turnaround time, tokens, and dollars, and when does CoEP's one-off build amortize? **RQ3 (Rule analysis and attribution):** which rules does the builder author and which does self-learning add, which recur across problems and fire most, and what does the LLM layer add over the shared framework?

Baselines **Refactory** (Hu et al. 2019) is the classical symbolic per-assignment tool: it refactors the class's correct solutions to match each buggy program's structure and transplants the divergent blocks. On its own corpus we use its published results; on Codeforces we run the released tool

in its strongest configuration. **PyDex** (Zhang et al. 2024) is per-submission LLM repair: each buggy submission is sent to the model together with the task description and tests, sampled candidates are filtered by the test oracle, and the surviving candidate closest to the student's program is returned. **PyDex$_{\mathbf{PC}}$** re-prices the same queries under provider prompt caching, as an upper bound that assumes a cache hit wherever the provider's rules allow one. **CoEP** is ours: a repairer synthesized once per assignment and model, evaluated in three configurations that form an ablation ladder. **CoEP$_{\mathbf{L}}$** runs the frozen local tool alone, with zero LLM use at deployment. **CoEP$_{\mathbf{LF}}$** adds a per-submission LLM fallback on abstentions. **CoEP$_{\mathbf{LFSL}}$** further absorbs successful fallback fixes into new rules.

Corpus	Problems	Buggy	Correct	Tests	LOC
Refactory	5	1,783	2,442	7.4	11.9
Codeforces	6	1,187	698	35.5	12.3
Total	11	2,970	3,140	22.7	12.0

Table 1: Dataset statistics. Tests is the average test-suite size per problem; LOC is the average number of non-blank lines per submission.

Datasets We evaluate on 11 problems and 2,970 buggy submissions drawn from two real-student corpora (Table 1). Refactory contributes five CS1-level assignments (Hu et al. 2019), the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). Codeforces contributes six competitive-programming problems from the open-r1 release (Penedo et al. 2025), built one per rating band across six difficulty levels from Newbie to Master (Codeforces); we keep only problems whose complete official test suites are released, so the oracle is the judge's own. A label audit executes every submission against its full suite, and submissions labeled buggy that already pass are excluded from all repair targets, leaving 1,686 truly buggy Refactory programs. No train/test split is used: every arm sees the same pools, as prior per-assignment systems assume, and all results additionally hold with CoEP's build samples excluded from the denominator.

LLM Configuration Every LLM arm runs on two models, one per provider: **GPT-5.4-mini** ($0.75/$4.50 per Mtok) and **Claude Sonnet 5** ($2/$10 per Mtok). We select each provider's cheapest tier at which prompt caching is functional, for two reasons. First, fairness to the caching baseline: PyDex$_{\mathrm{PC}}$ only competes if caching can engage, and the absolute cheapest tiers rule it out. We verified empirically that Claude Haiku 4.5 refuses to cache prompts below its 4,096-token minimum, above these repair prompts, and that GPT-5.4-nano's automatic caching did not engage even on identical 1,200-token prompts, whereas both selected models cache at their documented 1,024-token minimums. Second, cheap tiers keep every measured dollar gap a conservative floor, and the break-even N^* is price-invariant regardless (RQ2).

Evaluation Metrics Metrics are grouped by the research question they serve.

RQ1 (repair effectiveness). Repair rate (RR): percentage of buggy submissions for which the system produces a program passing the full test suite. **Relative patch size (RPS)**: tree-edit distance between buggy and repaired ASTs normalized by buggy program size, computed with the Refactory reference implementation (Hu et al. 2019); smaller patches preserve the student's approach and are pedagogically preferable, and RPS is *reported*, never used to gate a repair. Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests at the submission level, with per-assignment directions reported alongside pooled tests.

RQ2 (cost). Average turnaround time (ATT): mean wall-clock seconds to repair one submission end to end, decomposed into $\mathrm{ATT}_{\mathrm{cpu}}$ (local search on the CPU), $\mathrm{ATT}_{\mathrm{api}}$ (model-call latency), and their sum $\mathrm{ATT}_{\mathrm{all}}$; CoEP's one-off build wall-clock is reported separately as $\mathrm{ATT}_{\mathrm{build}}$ per assignment, an intercept like C_{build} rather than a per-submission term. **Input/output tokens (InT/OuT)**: mean prompt and response tokens per repaired submission. **LLM cost**: USD spent on model inference per repaired submission and per assignment (including a system's one-off build cost), priced at the tiers above from exact prompt/response token counts. The name is deliberate: monetary inference spend is only one of the costs this paper argues about, and latency, determinism, and data egress are carried by ATT and the setup. The **abstention rate** f, the fraction of submissions the local tool escalates, is what the break-even $N^* = C_{\mathrm{build}}/(\Delta(1-f))$ consumes.

RQ3 (rule analysis). The **repair-source split** attributes every fix to the mechanism that produced it (synthesized rule, generic rule, reference transplant, or LLM fallback). **Rule provenance** counts the rules the builder authored, the rules self-learning added, and the rules that actually fired at deployment, and tracks which recur across problems. **Diagnosis coverage** is the fraction of repaired submissions whose patch carries a misconception label (rule-fired repairs), quantifying how much of the instructor-facing profile the tool actually delivers rather than leaving it implicit.

Implementation Details All experiments run on a single Apple M1 Ultra workstation (64 GB RAM, macOS) under Python 3.13, CPU only for everything but the API calls. PyDex uses $K{=}10$ sampled candidates per submission, issued in parallel, with every call billed and timed. The three CoEP configurations are derived from a single pass over the submission stream with no duplicated LLM calls, so the reported local numbers are always the frozen tool's. Refactory runs with its original algorithm unchanged, ported to Python 3. We release the code, prompts, raw model outputs, and per-submission logs.

Results

We answer the three research questions stated in the Experimental Setup; together they cover the introduction's claims.

RQ1: Effectiveness and Patch Quality Table 2 presents the headline comparison across the setting grid. *[Cells are*

Model	Method	RR (%)	RPS	ATT (s)	InT/OuT	LLM Cost ($)
non-LLM	Refactory	–	–	–	–	
GPT-5.4 -mini	PyDex	–	–	–	–	
	PyDex_{PC}	–	–	–	–	
	CoEP_{L}	–	–	–	–	
	CoEP_{LF}	–	–	–	–	
	$\text{CoEP}_{\text{LFSL}}$	–	–	–	–	
Claude Sonnet 5	PyDex	–	–	–	–	
	PyDex_{PC}	–	–	–	–	
	CoEP_{L}	–	–	–	–	
	CoEP_{LF}	–	–	–	–	
	$\text{CoEP}_{\text{LFSL}}$	–	–	–	–	

Table 2: Main comparison on 2,970 buggy submissions: repair rate, mean relative patch size over repaired submissions, average turnaround time, mean input/output tokens, and USD per repaired submission (with build cost amortized). Refactory is model-independent; the LLM approaches are each run at the cheapest tier of both providers. $\text{CoEP}_{\text{L}}/_{\text{LF}}/_{\text{LFSL}}$ form an ablation ladder (local only / + fallback / + self-learning, absorption calls priced) and are derived from a single evaluation pass.

populated from the full run over all 2,970 submissions.] The central finding concerns CoEP's local repair against the strongest non-LLM tool. On the Refactory corpus of 1,686 buggy submissions, CoEP's local path of named rules, the shared generic library, and reference-guided transplant repairs **97.8%** of submissions at mean **0.326** RPS, exceeding Refactory's own reported **90.8%** at **0.40** on both axes, with higher coverage and smaller patches at zero marginal LLM cost. Per assignment the local path reaches 100% on four of the five and exceeds Refactory's per-problem repair rate on every assignment *[per-assignment table from the final validated run]*. This is the result that makes a local tool worth synthesizing. It is not merely cheaper than a symbolic tool that consumes every correct solution; it repairs more, and more surgically.

Against the per-submission LLM baselines, published results suggest functional repair is near-saturated on CS1-scale programs (Zhang et al. 2024), so we expect the separation in patch size and cost (RQ2) *[confirmed cells from the full run]*. Because CoEP's local patches are test-validated before a student sees them, its only local failure mode is abstention ($\perp$), never a wrong repair; abstentions take the per-submission LLM path, so the hybrid never sacrifices coverage.

RQ1. On the Refactory corpus CoEP's local repair reaches 97.8% RR at 0.326 mean RPS, beating the classical non-LLM tool (90.8%, 0.40) on both coverage and patch size, exceeding its repair rate on every assignment, at zero marginal LLM cost, while the hybrid escalates abstentions so coverage is never lost.

RQ2: Cost and Break-Even The three systems differ in *cost structure*, not just constant. A per-submission LLM arm (PyDex) pays an inference on every submission, so its cost grows linearly with enrollment and recurs every term; prompt caching (PyDex_{PC}) discounts the shared prefix but leaves the linear structure intact. CoEP pays a one-off synthesis cost C_{build} per assignment, then repairs the covered fraction at *zero* marginal LLM cost locally and pays a per-submission inference only on the abstaining tail.

Let Δ be the marginal per-submission saving of CoEP over a competitor and f the abstention (fallback) rate; synthesis wins once the enrollment exceeds $N^* = C_{\text{build}}/(\Delta(1-f))$. The expression is invariant to token price under the stated assumption that build and per-query tokens are billed on the same schedule: C_{build} and Δ then scale together, so a cheaper model or provider caching shrinks both sides equally and does not change N^*. The claim is about how many model calls per specification are made, not the price of a call, which is why cheap tiers do not blunt it. On the Refactory corpus the abstention rate is $f \approx 2.2\%$ (local RR 97.8%). *[Table 2's LLM-cost column, per-assignment f, and per-assignment N^* are populated from the full run.]* Synthesis is cheap: on the order of \$0.1 per assignment at these tiers, the price of a handful of baseline repairs, paid once *[exact per-model build costs from the full run]*. Because every arm runs at the cheapest tier, the measured dollar gaps are a *floor* on the effect, not its size. Projected with our measured prompt sizes onto a representative frontier tier of \$3/\$15 per Mtok, the class of GPT-5.4-pro or Claude Fable 5 deployments, the structure dominates at judge scale. Sending Codeforces' 12.6M archived submissions through a per-query repairer costs on the order of \$35K per pass and recurs with every cohort. The amortized route pays $\sim$\$2.5K of one-off builds for its 10,024 problems plus a $\sim$2% escalation tail of $\sim$\$0.8K. The gap is an order of magnitude and grows linearly with every further submission.

Beyond dollars, the covered fraction is repaired in about a second on a CPU with no network dependency, rate limits, or nondeterminism. A hosted per-query API offers none of these, and they matter most at deadline-night submission spikes.

RQ2. CoEP replaces a per-submission inference with a one-off cents-scale synthesis plus a fallback-only tail, breaking even at $N^* = C_{\text{build}}/(\Delta(1-f))$; the bound is price-invariant, so cheaper models and caching shrink both sides without changing when amortization wins.

RQ3: Rule Analysis and Mechanism Attribution **Two complementary local mechanisms.** CoEP's local repair combines named rules (problem-specific plus the shared generic library) with reference-guided transplant, and the two carry different loads. The rules make small, labelled edits for the common misconceptions, such as a relaxed comparison or an off-by-one bound, at very small patch size (mean RPS $\approx$0.03). But most student bugs are not single-token slips; they need a medium, structural edit no token-level rule expresses, and there the reference-guided transplant supplies it by adopting the smallest set of differing hunks from a structurally-matched correct program, variable-renamed to fit. Across the Refactory corpus the rule layer fixes $\sim$13% of repaired submissions (220/1,648) and the transplant the rest; the transplant is what lifts a rule-only tool (which caps near 18%) to competitive coverage, while the rules keep the com-

mon cases at near-minimal patch size and carry the diagnosis below. A *framework-only* ablation, the identical tool with the LLM-authored rules removed, isolates the LLM layer: on this corpus it adds nothing to coverage and almost nothing to patch size (RR 97.75% vs. 97.75%; mean RPS 0.326 vs. 0.329), because the transplant absorbs the cases the rules handled, and the always-on generic library accounts for two-thirds of the rule-layer firings (143 of 220). What the build measurably buys here is therefore not coverage but diagnosis, the misconception-labeled repairs, plus the strategy-matched escalation path; whether LLM-authored rules buy coverage where correct pools are sparse is precisely what the harder Codeforces half of our benchmark tests *[Codeforces cells from the full run]*.

Diagnosis for free. Every rule-produced patch carries the ordered rule names that produced it, so aggregated over a class the tool yields a misconception profile: how often each named mistake occurred, and on which programs. An instructor gets this report at no extra cost, and a per-submission API never emits it. Diagnosis coverage, the fraction of local repairs carrying a misconception label, is about 13% on this corpus, concentrated on the most frequent slips. This is a property of the artifact, not the coverage number: it is why a local, auditable repairer is worth synthesizing even where a hosted model would pass the same tests.

What is authored, what is learned, what recurs. Every repair event logs the ordered rule names that produced it, split by origin (builder-authored vs. self-learned vs. generic), so we can report which misconceptions the builder anticipated, which only surfaced as fallback fixes that self-learning absorbed, which rules recur across problems (the generic comparison/bound slips), and which fire most often per assignment. *[Rule-provenance tables from the full run's event logs.]*

Builder capability. Synthesis is run with the same two models as every LLM arm (GPT-5.4-mini, Claude Sonnet 5). Showing the builder *worked (buggy → fixed) pairs* rather than raw buggy samples turns "infer the fix, then generalize" into "generalize this shown repair"; in earlier probes with the absolute cheapest tiers this lifted build-sample coverage from single digits to 75%, and the shared generic library plus the reference engine make the local path robust to builder weakness regardless. *[Per-builder build-sample coverage and rule quality from the full run.]* Capability is bought once per assignment, not once per submission; that is the entire point.

RQ3. Named rules (RPS ≈ 0.03, with a misconception label) cover the common slips; reference-guided transplant supplies the structural edits that lift coverage from a rule-only $\approx 18\%$ to Refactory-beating, and the rule provenance yields a per-class diagnosis a hosted API cannot.

Related Work

APR for programming assignments. The pre-LLM systems surveyed in the introduction (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022) exploit per-assignment structure but need many prior solutions; our transplant stage stands openly in their reference-reuse lineage, at hunk rather than block granularity, and our head-to-head run shows the composed artifact exceeding the strongest of them on both coverage and patch size. Closest to our synthesis layer, Refazer learns per-assignment syntactic transformations from (buggy, fixed) student pairs with PROSE (Rolim et al. 2017); CoEP replaces the example-driven synthesizer with a cheapest-tier LLM that *names* the misconception behind each rule, and surrounds the learned layer with generic search, reference transplant, and harness-gated escalation. At AAAI the setting stays active from the per-submission side: counterexample-guided and preference-tuned minimal repair at AAAI-25 (Orvalho, Janota, and Manquinho 2025; Dai et al. 2025), and AAAI-26's learner-tailored repair, which retrieves per-assignment example fixes to guide each query (Dai et al. 2026), amortizing *data* into a retrieval index while every repair still pays for an LLM call (our PyDex arm bounds this design's economics). CoEP instead removes the per-submission call for everything its artifact covers and pays only on escalation.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Templates, cost-efficient inference, and tool synthesis. PAR and TBar showed a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic layer is a mutation-inversion instance (Just et al. 2014), and its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM. FrugalGPT cascades, SRepair's cheap pipelines, and provider caching (Chen, Zaharia, and Zou 2024; Xiang et al. 2024; Anthropic 2025; OpenAI 2024) all optimize *within* the pay-per-query paradigm and compose with CoEP, which instead removes queries. Voyager and LLMs-as-tool-makers amortize agent-written code across episodes (Wang et al. 2024; Cai et al. 2024), self-refinement improving one-shot generations (Madaan et al. 2023; Shinn et al. 2023); CoEP adds what education forces: verifiability (never-trust-self-reports) and minimal, measured patches, because its output is feedback to a learner.

Ethical Statement

CoEP automates feedback, not grading, under instructor oversight; unhandled students get full-LLM treatment; repairs are auditable; data are public.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-

claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Codeforces. ???? Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert, ...

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Figure 1 and the Approach section's stage specification). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the break-even analysis in RQ2 is marked as derivation over measured quantities). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use two publicly available corpora: the Refactory dataset (Hu et al. 2019) and Codeforces problems and submissions from the open-r1 release (Penedo et al. 2025). Our scripted import pipeline (test parsing, per-test entry points, oracle-fidelity audit, rating-band problem selection) is released, so the exact evaluation sets are reconstructible from the public corpora. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample per call), execution timeouts, and the no-split whole-pool protocol are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic. Builder synthesis is stochastic; every round's prompt, response, and build-sample outcome for both builder tiers is released with the artifacts.